

Contents

SQL Efficiency Prompt Chain	1
Use Case	1
Required Local Preparation	1
Prompt Sequence	1
Related Procedure Closet	1
Related Reference Drawers	2
SQL Efficiency Chain - 01 Requirement Intake	2
SQL Efficiency Chain - 02 Logic Map	2
SQL Efficiency Chain - 03 SQL Draft	3
SQL Efficiency Chain - 04 Efficiency Review	3
SQL Efficiency Chain - 05 Final Query And Checks	3

SQL Efficiency Prompt Chain

Use Case

Use this chain when a human analyst needs Copilot to turn a business requirement into efficient Oracle SQL. The chain makes Copilot map business logic first, then draft SQL only after the mapping is approved.

This is not an EXPLAIN PLAN tuning workflow. Use `procedures/query_tuner/SKILL.md` when a slow query already exists and plan evidence is available.

Required Local Preparation

- Business objective and expected output grain.
- Candidate source tables and any known join keys.
- Required output fields, filters, aggregations, and date windows.
- Any known row-volume, partition, indexing, or performance constraints.
- Validation checks that prove the SQL matches the business requirement.

Prompt Sequence

1. `01-requirement-intake.md`: restate objective, tables, fields, joins, filters, aggregations, and output grain.
2. `02-logic-map.md`: produce the required business-rule logic map table.
3. `03-sql-draft.md`: draft Oracle SQL only after the logic map is approved.
4. `04-efficiency-review.md`: review the draft for avoidable inefficiency.
5. `05-final-query-and-checks.md`: produce final SQL, efficiency explanation, and validation checklist.

Related Procedure Closet

- `procedures/query_tuner/SKILL.md` for separate plan-based tuning workflows.

Related Reference Drawers

- [references/oracle-sql/sections/oracle_idioms.md](#)
- [references/oracle-sql/sections/query_tuning.md](#)

SQL Efficiency Chain - 01 Requirement Intake

Act as a senior Oracle SQL developer converting a business requirement into an efficient query. This is step 1 of a multi-turn workflow.

Do not write SQL yet.

First, restate the requirement in this structure:

1. Business objective.
2. Source tables.
3. Required fields.
4. Joins.
5. Filters.
6. Aggregations.
7. Expected output grain.

Then list only the missing inputs that block a correct logic map. Ask targeted questions about ambiguous business rules, source tables, join keys, filters, date windows, aggregation grain, and validation expectations.

When I answer, wait for the next prompt before writing SQL.

SQL Efficiency Chain - 02 Logic Map

Use my requirement details to create a business logic map before writing SQL.

Do not write SQL yet.

Return exactly this table format:

business rule	source table	source field	join/filter logic	transformation	output field	validation check
---------------	--------------	--------------	-------------------	----------------	--------------	------------------

Rules:

1. Include one row per business rule, output field, join rule, filter rule, or aggregation rule.
2. Make the expected output grain explicit.
3. Mark any uncertain source field, join, filter, or transformation as **needs confirmation**.
4. Include validation checks for row counts, null handling, duplicate risk, aggregation totals, and date/filter boundaries where relevant.
5. End with: **Approve this logic map before I draft SQL.**

After producing the table, stop. Do not draft SQL until I explicitly approve the logic map.

SQL Efficiency Chain - 03 SQL Draft

The logic map is approved. Now draft Oracle SQL from the approved map.

Use Oracle-efficient structures:

1. Apply selective predicates as early as practical.
2. Use explicit joins with clear join conditions.
3. Use CTEs for readable staged logic when they clarify filters, deduplication, aggregation, or joins.
4. Avoid unnecessary `DISTINCT`; explain any `DISTINCT` that remains.
5. Avoid functions on indexed filter columns where possible.
6. Keep aggregation grain clear and aligned with the approved output grain.

Return:

1. The SQL draft.
2. A short mapping from each CTE or query block to the approved logic map.
3. Assumptions still present in the SQL.
4. Questions that must be resolved before finalizing, if any.

Do not perform `EXPLAIN PLAN` tuning in this step. If plan evidence is needed, say that this belongs in the query tuning workflow.

SQL Efficiency Chain - 04 Efficiency Review

Review the SQL draft for correctness and avoidable inefficiency before final delivery.

Use this review checklist:

1. Business objective still matches the approved logic map.
2. Source tables and joins match the approved source mapping.
3. Filters are selective and applied at the right stage.
4. Indexed filter columns are not wrapped in avoidable functions.
5. Aggregation grain is explicit and does not multiply rows.
6. `DISTINCT` is absent or justified by a specific duplicate source.
7. CTEs improve staged readability without hiding unnecessary scans.
8. Output fields are named clearly and trace back to business rules.
9. Validation checks can prove row count, totals, duplicates, and date bounds.

Return:

1. Issues found, ordered by severity.
2. Revised SQL if changes are needed.
3. Remaining risks or assumptions.
4. Whether the query is ready for final packaging.

Do not ask for `EXPLAIN PLAN` unless I am switching to the query tuning workflow.

SQL Efficiency Chain - 05 Final Query And Checks

Prepare the final Oracle SQL and handoff notes from the reviewed draft.

Return:

1. Final SQL.
2. Business objective summary.
3. Logic-map coverage summary.
4. Explanation of why the SQL is efficient.
5. Validation checklist.
6. Known assumptions and follow-ups.

The efficiency explanation must cover:

1. Selective predicates and where they are applied.
2. Join strategy clarity.
3. CTE purpose and staged logic.
4. Why unnecessary `DISTINCT` is avoided or why any remaining one is needed.
5. How indexed filter columns are protected from avoidable function wrapping.
6. How the aggregation grain is controlled.

The validation checklist must include:

1. Row-count checks at the expected output grain.
2. Duplicate checks on the output key.
3. Null checks for required output fields.
4. Aggregate reconciliation checks.
5. Date/filter boundary checks.
6. Spot checks tying output fields back to source records.

Keep the answer concise enough to paste into a SQL work item or review thread.